

Arquitectura de SatsPath en Arkade Wallet

El protocolo de **SatsPath** está completamente integrado en la wallet a través de un puente con WebAssembly (WASM). Esto permite que toda la criptografía pesada y la lógica de ruteo y resolución ocurran de forma ultra-rápida y segura del lado del cliente, utilizando el mismo código robusto en Rust de **satspath-core**.

A continuación te presento un diagrama de secuencia de cómo fluyen los datos en las dos etapas principales: el **Flujo del Receptor** (creación de identidad) y el **Flujo del Emisor** (resolución y envío).

sequenceDiagram

```
participant User as Usuario
participant Wallet as Arkade Wallet (React)
participant Bridge as satspath.ts (WASM Bridge)
participant WASM as satspath-wasm (Rust)
participant Network as Red (Nostr / DNS)
```

%% Flujo del Receptor

```
rect rgb(35, 20, 50)
```

Note over User, Network: 1. FLUJO DEL RECEPTOR (Crear Identidad)

User->>Wallet: Escribe Alias y Direcciones (Settings)

Wallet->>Wallet: Detona animación de Láseres

Wallet->>Bridge: buildSatsPathProfile()

Bridge->>WASM: generate_identity_keypair()

Note right of WASM: Genera llave secp256k1

WASM-->>Bridge: IdentityKeypair (Hex)

Bridge->>Bridge: Construye Perfil JSON crudo

Bridge->>WASM: sign_profile_json(json, secretKey)

Note right of WASM: Aplica hash y firma de Schnorr

WASM-->>Bridge: Firma Criptográfica

Bridge-->>Wallet: SignedPaymentProfile

Wallet->>Wallet: Guarda en LocalStorage

Wallet->>User: Muestra JSON firmado listo para la red

```
end
```

%% Flujo del Emisor

```
rect rgb(50, 30, 20)
```

Note over User, Network: 2. FLUJO DEL EMISOR (Enviar Sats)

User->>Wallet: Enviar a "chelo@dev.idk"

Wallet->>Bridge: routePayment()

Bridge->>WASM: quote()

WASM->>Network: Resuelve Perfil (BIP353/NIP05/Local)

Network-->>WASM: SignedPaymentProfile

Note right of WASM: Valida firmas y expiración

WASM->>WASM: router::select_route()

Note right of WASM: Sistema de Scoring elige
el método más barato/óptimo

WASM-->>Bridge: QuoteResponse (Mejor método)

Bridge-->>Wallet: RouteResult (Ej. Lightning)

Wallet->>User: Muestra pantalla de confirmación

```
end
```

Componentes Clave

1. **Arkade Wallet (React):** Es la capa de presentación. Maneja las animaciones (como los láseres), la recolección de datos del usuario, el guardado local y muestra los resultados finales.
2. **satspath.ts (WASM Bridge):** Es el traductor. Se encarga de convertir los objetos de JavaScript a

- los tipos primitivos que espera Rust, y viceversa. Aquí es donde inyectamos los “mocks” para desarrollo.
3. **satspath-wasm (Rust):** El cerebro de la operación. Hereda el código de **satspath-core**. Contiene el generador criptográfico seguro, la lógica matemática de las firmas de Schnorr, y el motor de Scoring que decide si es más barato irse por Ark, Lightning o On-chain dependiendo de las comisiones en tiempo real.

Visión: Arquitectura Soberana P2P

Este es el modelo teórico 100% cypherpunk que busca eliminar la dependencia de servidores DNS y HTTP, garantizando privacidad mediante conexiones enjambre y Web of Trust.

```
graph TD
    %% Flujo de Usuarios
    User((Usuario)) -- "Enviar Sats" --> Bob((Bob))
    Bob -- "Recibir Sats" --> QR["Muestra Código QR"]
    QR --> Alias["QR contiene Alias P2P"]
    Alias --> Address["bob@satspath.p2p"]
    Address --> PubKeys["Contiene Identity Pubkey"]

    %% Protocolos y Capas
    subgraph Capas de Red
        LN["Lightning Network"]
        ONCHAIN["On-chain"]
        ARK["Ark Protocol"]
        P2P["Red P2P (Hyperswarm)"]
    end

    end

    %% Seguridad Soberana
    subgraph Security [Seguridad Soberana y Confianza P2P]
        Sec["Seguridad"]
        Enc["Cifrado P2P (ECDH)"]
        Id["Identidad Criptográfica (Pubkey secp256k1)"]

        %% Reemplazamos Email/Phone por métodos descentralizados
        QR_Scan["Verificación Out-of-band (QR Scan)"]
        WoT["Web of Trust (Gráfica Social Nostr)"]
        Hashcash["Prueba de Trabajo (Anti-Spam)"]

        Trust["Confianza Matemática y Social"]

        Sec --> Enc
        Sec --> Id
        Sec --> QR_Scan
        Sec --> WoT
        Sec --> Hashcash

        QR_Scan --> Trust
        WoT --> Trust
        Hashcash --> Trust
    end

    end
```

Arquitectura Actual (Implementada en el Código)

A diferencia del diagrama teórico 100% soberano que discutimos arriba, el código actual de **satspath-core** y **satspath-router** implementa una arquitectura híbrida de transición.

Actualmente, el sistema resuelve los alias apoyándose en infraestructuras existentes (DNS y HTTP) y tiene un prototipo básico de P2P con Pear/Hyperswarm, pero **aún no implementa** el cifrado ECDH, los Handshakes ni las pruebas anti-spam (Hashcash).

Aquí tienes el diagrama exacto de cómo funciona el código **hoy**:

```
graph TD
    %% Flujo de Usuarios
    User((Usuario)) -- "Enviar Sats a chelo@dev.idk" --> Wallet[(Arkade Wallet)]
    Wallet -- "routePayment()" --> Bridge[satspath.ts WASM Bridge]
    Bridge -- "quote()" --> Router["SatsPath Router (Rust)"]

    %% Resolvers Actuales Implementados
    subgraph Resolvers [Módulos de Resolución Actuales]
        DNS["BIP353Resolver (DNS TXT)"]
        Nostr["NostrResolver (HTTP NIP-05)"]
        Pear["PearResolver (P2P Básico)"]
        Local["MockResolver (Desarrollo)"]
    end

    Router --> DNS
    Router --> Nostr
    Router --> Pear
    Router --> Local

    %% Detalle de los Resolvers
    DNS -. "Consulta DNS" .-> Cloudflare[Servidor DNS]
    Nostr -. "GET /.well-known/nostr.json" .-> WebServer[Servidor Web]

    %% Detalle del fallo de soberanía actual
    Pear -. "node satspath-pear/index.js resolve <alias>" .-> NodeJS[Script Local Node.js]
    NodeJS -. "SHA256(alias) = Topic" .-> Hyperswarm[Hyperswarm DHT]

    %% Proceso de Scoring
    Cloudflare -. "SignedPaymentProfile" .-> Router
    WebServer -. "SignedPaymentProfile" .-> Router
    Hyperswarm -. "SignedPaymentProfile" .-> Router

    Router -- "router::select_route()" --> Scoring[Motor de Scoring]
    Scoring -- "QuoteResponse" --> Wallet
```

Implementation Plan: Migración a P2P Soberano (Trusted Connections)

Para lograr que el código de SatsPath cumpla con el modelo 100% soberano (y resolver el problema de privacidad del alias expuesto), se debe ejecutar el siguiente plan de desarrollo:

Fase 1: Enjambres Ofuscados y Cifrado (ECDH)

- ☐ **Derivación de Tópico Seguro:** Modificar **satspath-core** para que no pase el alias en texto plano al script de Node.

- ☐ **ECDH Compartido:** Implementar Diffie-Hellman usando la `IdentityPubkey` del emisor y receptor para generar un secreto. El `Topic` de Hyperswarm será el `SHA256(SharedSecret)`.
- ☐ **Cifrado de Carga Útil:** Usar ChaCha20-Poly1305 (con el secreto compartido) para encriptar los paquetes de datos P2P, garantizando confidencialidad total.

Fase 2: Protocolo TCP-Style (SYN / SYN-ACK)

- ☐ **Paquete SYN (Emisor -> Receptor):** El nodo de Hyperswarm emisor debe enviar un payload cifrado con su intención de pago (`Amount` y `Nonce` aleatorio), y firmado criptográficamente por la `Wallet`.
- ☐ **Generación Just-In-Time (Ark):** El nodo receptor recibe el SYN, valida la firma, y detona un llamado al servidor ASP de Ark para generar un VTXO dinámico que se bloquee específicamente para esta transacción.
- ☐ **Paquete SYN-ACK (Receptor -> Emisor):** El receptor devuelve el invoice fresco, su alias local, y firma el `Nonce` para probar liveness.

Fase 3: Escudos Anti-Bot / Anti-IA (Web of Trust)

- ☐ **Módulo de Web of Trust (WoT):** Integrar Nostr (NIP-65) en la wallet para mantener una gráfica social de llaves públicas conocidas.
- ☐ **Reglas de Aceptación:** Configurar el demonio P2P para rechazar/dropear automáticamente conexiones SYN de Pubkeys que estén a más de 2 grados de separación en la red de confianza.
- ☐ **Integración de Hashcash (Lightning/Ark):** Obligar a conexiones fuera de la WoT a adjuntar un HTLC válido por una micro-transacción de 10 sats para cubrir el costo computacional del ataque Sybil.